Backpropagation in One MLP Layer: From a Three-Dimensional Tensor to an Outer Product

N-20250926

§1.1 The weight derivative and its three indices

Consider one middle layer of a multilayer perceptron:

$$z = Wx + b, \qquad y = \sigma(z).$$

Here

$$x \in \mathbb{R}^n, \qquad W \in \mathbb{R}^{m\times n}, \qquad b \in \mathbb{R}^m, \qquad z, y \in \mathbb{R}^m.$$

The activation function σ acts elementwise, for example sigmoid or ReLU. The goal of backpropagation is to compute

$$\frac{\partial L}{\partial W}$$

for a loss function L.

The organization of derivatives as linear maps and reverse-mode pullbacks follows MIT OCW, *Matrix Calculus for Machine Learning and Beyond*.[1]

The softmax–cross-entropy setting provides a standard example. If the final layer produces logits z_i, then

$$\hat{y}_i = \frac{e^{z_i}}{\sum_j e^{z_j}}, \qquad L = -\sum_i t_i \ln \hat{y}_i,$$

where t_i is a one-hot label. Backpropagation asks how this scalar loss changes when each weight $W_{j,k}$ changes.

Strictly speaking, the derivative of the vector $y \in \mathbb{R}^m$ with respect to the matrix $W \in \mathbb{R}^{m\times n}$ has three indices:

$$\frac{\partial y_i}{\partial W_{j,k}}.$$

The index i chooses the output component, j chooses the row of W, and k chooses the column of W. So the object has shape

$$m \times m \times n.$$

This is the source of the confusion recorded in the note. The derivative is theoretically a third-order tensor, but in neural-network code one usually sees only a matrix gradient. The matrix appears after chain-rule contraction with the upstream gradient.

Write the linear preactivation component as

$$z_r = \sum_{t=1}^{n} W_{r,t} x_t + b_r.$$

[1] MIT 18.S096 lecture notes.

Then

$$\frac{\partial z_r}{\partial W_{j,k}} = \begin{cases} x_k, & r = j, \\ 0, & r \neq j, \end{cases} = \delta_{rj} x_k.$$

This is sparse because z_r depends only on row r of W.

For $y_i = \sigma(z_i)$ with elementwise activation,

$$\frac{\partial y_i}{\partial z_r} = 0 \quad (i \neq r), \qquad \frac{\partial y_i}{\partial z_i} = \sigma'(z_i).$$

Thus

$$\frac{\partial y_i}{\partial W_{j,k}} = \sum_{r=1}^{m} \frac{\partial y_i}{\partial z_r} \frac{\partial z_r}{\partial W_{j,k}} = \frac{\partial y_i}{\partial z_j} x_k.$$

For elementwise activation this is nonzero only when $i = j$.

§1.2 Scalar losses contract the tensor

The loss L is scalar. Therefore

$$\frac{\partial L}{\partial W_{j,k}} = \sum_{i=1}^{m} \frac{\partial L}{\partial y_i} \frac{\partial y_i}{\partial W_{j,k}}.$$

Substituting the previous formula gives

$$\frac{\partial L}{\partial W_{j,k}} = \sum_{i=1}^{m} \frac{\partial L}{\partial y_i} \frac{\partial y_i}{\partial z_j} x_k.$$

The sum over i is exactly the j-th component of the upstream gradient with respect to z:

$$\delta_j = \frac{\partial L}{\partial z_j}.$$

Hence

$$\frac{\partial L}{\partial W_{j,k}} = \delta_j x_k.$$

Stacking all j, k gives the outer product

$$\boxed{\frac{\partial L}{\partial W} = \delta x^T}$$

where

$$\delta = \frac{\partial L}{\partial z}.$$

The original derivative $\partial y / \partial W$ is a tensor with shape (m, m, n). But backpropagation does not need to store it. The chain rule contracts it with the vector $\partial L / \partial y$, leaving a matrix of shape (m, n).

Equivalently:

$$\text{three-dimensional tensor} \xrightarrow{\text{chain-rule summation}} \text{matrix gradient}.$$

This is not a loss of information for gradient descent, because gradient descent only needs the derivative of the scalar loss with respect to each parameter.

For the final layer with softmax and cross entropy, the derivative simplifies to

$$\frac{\partial L}{\partial z} = \hat{y} - t.$$

Therefore the final-layer weight gradient is

$$\frac{\partial L}{\partial W} = (\hat{y} - t)x^T.$$

This is why implementation code often looks much simpler than the theoretical tensor derivation.

§1.3 Reverse-mode derivatives as cotangent pullbacks

The same computation can be described as a pullback of covectors. A forward layer is a map

$$f : \text{parameters and activations} \longrightarrow \text{outputs}.$$

Its differential sends small forward perturbations to small output perturbations:

$$df : T_x X \to T_{f(x)} Y.$$

But the loss supplies a covector at the output,

$$dL \in T^*_{f(x)} Y,$$

and backpropagation sends it backward by precomposition:

$$f^*(dL) = dL \circ df \in T^*_x X.$$

This is why reverse-mode differentiation is naturally contravariant.

For a batched linear layer in PyTorch convention,

$$X \in \mathbb{R}^{b\times n}, \qquad W \in \mathbb{R}^{m\times n}, \qquad Y = XW^T \in \mathbb{R}^{b\times m},$$

the component formula is

$$Y_{\alpha i} = \sum_{k=1}^{n} X_{\alpha k} W_{ik}.$$

Hence

$$\frac{\partial Y_{\alpha i}}{\partial W_{jk}} = \delta_{ij} X_{\alpha k},$$

a Jacobian tensor of shape (b, m, m, n). Contracting with the upstream gradient $G = \partial L/\partial Y$, of shape (b, m), gives

$$\begin{aligned}\frac{\partial L}{\partial W_{jk}} &= \sum_{\alpha,i} G_{\alpha i} \frac{\partial Y_{\alpha i}}{\partial W_{jk}} \\ &= \sum_{\alpha} G_{\alpha j} X_{\alpha k},\end{aligned}$$

so

$$\frac{\partial L}{\partial W} = G^T X.$$

If the weights are stored instead as $W_{\text{col}} \in \mathbb{R}^{n\times m}$ and $Y = XW_{\text{col}}$, the same formula appears as

$$\frac{\partial L}{\partial W_{\text{col}}} = X^T G.$$

The two expressions differ only by the convention for storing the weight matrix.

§1.4 A numerical gradient and a useful consistency check

Take $m = 2$, $n = 3$, input $x = (1, 2, 3)^T$, and preactivations $z = (0.1, -1)^T$. For sigmoid activation σ,

$$\sigma'(0.1) \approx 0.2494, \qquad \sigma'(-1) \approx 0.1966.$$

The matrix obtained by stacking the derivatives of each output y_i with respect to the entries in *its own row* $W_{i,:}$ is

$$\begin{pmatrix} 0.2494 & 0.4988 & 0.7482 \\ 0.1966 & 0.3932 & 0.5898 \end{pmatrix}.$$

This is a collection of two nonzero slices of the full (m, m, n) Jacobian, *not* the complete Jacobian itself. If the upstream loss gradient happens to be $\partial L/\partial y = (1, 1)^T$, the contracted *weight gradient* is numerically the same matrix. For a general upstream gradient u, however, it becomes

$$\frac{\partial L}{\partial W} = (u \odot \sigma'(z))x^T.$$

This distinction explains when the numerical outer product is a legitimate gradient and when it merely displays the local activation sensitivities.

In vectorized notation one may write $\mathrm{vec}(\delta x^T) = x \otimes \delta$ when columns are stacked, but constructing this Kronecker product gains nothing computationally. For batches the correct matrix formula in the convention $X \in \mathbb{R}^{b\times n}$, $G \in \mathbb{R}^{b\times m}$, $Y = XW^T$, is $\nabla_W L = G^T X$; its transpose $X^T G$ belongs to the alternative column-stored weight convention.